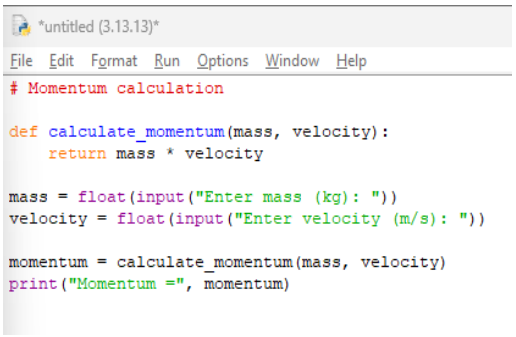
PRACTICAL:Perform following experiments using python programming language.

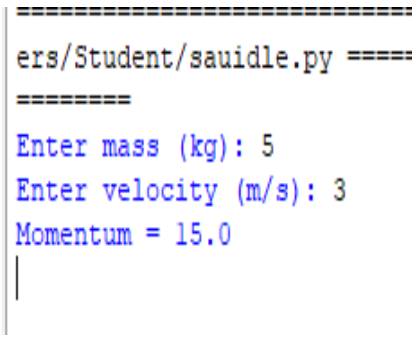
NAME	Saurabh Dariba Godase
CLASS	SE2 /S23
PRN	202501100167
SUBJECT	Essential of Data Science

PRACTICAL NO.01

Practice Lab Assignments: 1.

To accept an object mass in kilograms and velocity in meters per second and display its momentum. Momentum is calculated as e=mc2 where m is the mass of the object and c is its velocity.



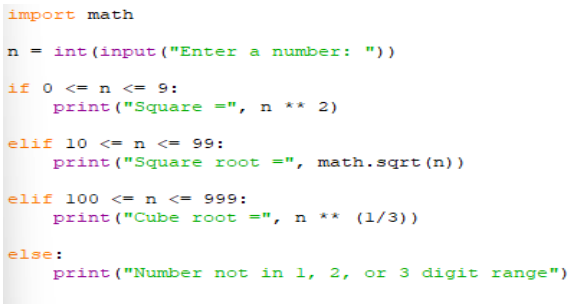


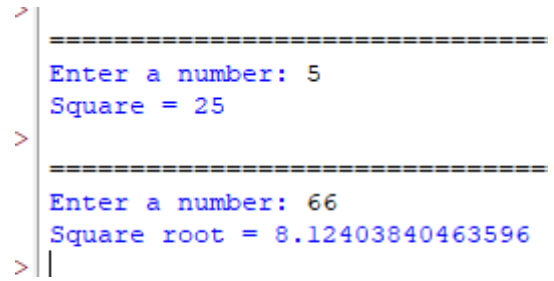
2. Write a Python program for following conditions.

ˆ If n is single digit print square of it.

ˆ If n is two digit print square root of it.

ˆ If n is three digit print cube root of it.





3. Read the birth date and salary in rupees of employees. Perform data transformation for birthdate to age and also salary which is in rupees to salary in dollars using functions. 4. Print the reverse number of a given number

```python
from datetime import datetime

def calculate_age(birth_year):
    return datetime.now().year - birth_year

def convert_salary_rupees_to_dollars(salary_rupees):
    exchange_rate = 83  # Approx value
    return salary_rupees / exchange_rate

birth_year = int(input("Enter birth year: "))
salary_rupees = float(input("Enter salary in rupees: "))

age = calculate_age(birth_year)
salary_dollars = convert_salary_rupees_to_dollars(salary_rupees)

print("Age =", age)
print("Salary in dollars =", salary_dollars)
```

```
Enter birth year: 2006
Enter salary in rupees: 78677
Age = 20
Salary in dollars = 947.9156626506024

==================================== RES
Enter birth year: 2007
Enter salary in rupees: 56555
Age = 19
Salary in dollars = 681.3855421686746
```

Lab Assignment: 1. To accept students five courses marks and compute his/her result. Student is passing if he/she scores marks equal to and above 40 in each course. If student scores aggregate greater than 75 percentage, then the grade is distinction. If aggregate is greater than or equal to 60 and less than 75 then the grade if first division. If aggregate is greater than or equal 50 and less than 60, then the grade is second division. If aggregate is greater than or equal 40 and less than 50, then the grade is third division.

```python
marks = []
total = 0
pass_flag = True

print("Enter marks for 5 subjects:")

for i in range(5):
    m = float(input(f"Subject {i+1}: "))
    marks.append(m)
    total += m
    if m < 40:
        pass_flag = False

percentage = total / 5

print("\nTotal =", total)
print("Percentage =", percentage)

if pass_flag:
    print("Result: PASS")

    if percentage > 75:
        print("Grade: Distinction")
    elif 60 <= percentage < 75:
        print("Grade: First Division")
    elif 50 <= percentage < 60:
        print("Grade: Second Division")
    elif 40 <= percentage < 50:
        print("Grade: Third Division")
else:
    print("Result: FAIL")
```

```
Enter marks for 5 subjects:
Subject 1: 98
Subject 2: 89
Subject 3: 77
Subject 4: 88
Subject 5: 99

Total = 451.0
Percentage = 90.2
Result: PASS
Grade: Distinction
```

2. Solve the Fibonacci sequence using recursive function in Python.

```
# Fibonacci using recursion

def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n = int(input("Enter number of terms: "))

print("Fibonacci sequence:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

```
Enter number of terms: 10
Fibonacci sequence:
0 1 1 2 3 5 8 13 21 34
```

3. Write a Python program to print different patterns.

```
File  Edit  Format  Run  Options  Window  Help
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    print("*" * i)
```

```
==================================
Enter number of rows: 5
*
**
***
****
*****
```

Perform all statistical analysis (Average, Max, Min, Sum, on emplyoee data.

```
File  Edit  Format  Run  Options  Window  Help
rows = int(input("Enter number of rows: "))

for i in range(rows, 0, -1):
    print("*" * i)
```

```
==================================== I
Enter number of employees: 5
Enter salary of employee 1: 50000
Enter salary of employee 2: 500000
Enter salary of employee 3: 25000
Enter salary of employee 4: 30000
Enter salary of employee 5: 400000

--- Statistical Analysis ---
Total Salary = 1005000.0
Average Salary = 201000.0
Maximum Salary = 500000.0
Minimum Salary = 25000.0
```

PRACTICAL NO.02

Practice Lab Assignment:

1. Perform all List Operations, Dictionary Operations

File Edit Format Run Options Window

```
# List operations

lst = [10, 20, 30, 40]

print("Original List:", lst)

# Add element
lst.append(50)
print("After append:", lst)

# Insert element
lst.insert(2, 25)
print("After insert:", lst)

# Remove element
lst.remove(20)
print("After remove:", lst)

# Pop element
lst.pop()
print("After pop:", lst)

# Sort list
lst.sort()
print("Sorted List:", lst)

# Reverse list
lst.reverse()
print("Reversed List:", lst)
```

```
========== RE

Original List: [10, 20, 30, 40]
After append: [10, 20, 30, 40, 50]
After insert: [10, 20, 25, 30, 40, 50]
After remove: [10, 25, 30, 40, 50]
After pop: [10, 25, 30, 40]
Sorted List: [10, 25, 30, 40]
Reversed List: [40, 30, 25, 10]
```

Lab Assignment

: 1. Select the number from the entered list and find its position in Python (use Linear Search).

```
# Cricket Team using Dictionary

players = {
    "Virat": 175,
    "Rohit": 170,
    "Gill": 178,
    "Rahul": 180,
    "Hardik": 183,
    "Jadeja": 173,
    "Bumrah": 181,
    "Shami": 177,
    "Surya": 176,
    "Ishan": 168,
    "Siraj": 182
}

# Find tallest player
captain = max(players, key=players.get)

print("Team Players and Heights:")
for name, height in players.items():
    print(name, ":", height, "cm")

print("\nCaptain (Tallest Player):", captain)
print("Height:", players[captain], "cm")
```

```
Team Players and Heights:
Virat : 175 cm
Rohit : 170 cm
Gill : 178 cm
Rahul : 180 cm
Hardik : 183 cm
Jadeja : 173 cm
Bumrah : 181 cm
Shami : 177 cm
Surya : 176 cm
Ishan : 168 cm
Siraj : 182 cm

Captain (Tallest Player): Hardik
Height: 183 cm
```

Self Study Assignment:

1. Consider the student subjects marks are stored in the list. Find the total marks and percentage of students. 2. Perform all tuple operations.

```
marks = [78, 85, 90, 88, 76]

total = sum(marks)
percentage = total / len(marks)

print("Marks:", marks)
print("Total Marks:", total)
print("Percentage:", percentage, "%")
```

```
Marks: [78, 85, 90, 88, 76]
Total Marks: 417
Percentage: 83.4 %
```

PRACTICAL NO.03

Practice Lab Assignment:

1. Perform all the Numpy operations in python.

```python
# Tuple Operations

t = (10, 20, 30, 40, 50)

print("Tuple:", t)

# Length
print("Length:", len(t))

# Indexing
print("First Element:", t[0])

# Slicing
print("Slice:", t[1:4])

# Concatenation
t2 = (60, 70)
print("Concatenation:", t + t2)

# Repetition
print("Repetition:", t * 2)

# Membership
print("Is 30 present?", 30 in t)

# Iteration
print("Tuple Elements:")
for item in t:
    print(item)
```

```
Tuple: (10, 20, 30, 40, 50)
Length: 5
First Element: 10
Slice: (20, 30, 40)
Concatenation: (10, 20, 30, 40, 50, 60, 70)
Repetition: (10, 20, 30, 40, 50, 10, 20, 30, 40, 50)
Is 30 present? True
Tuple Elements:
10
20
30
40
50
```

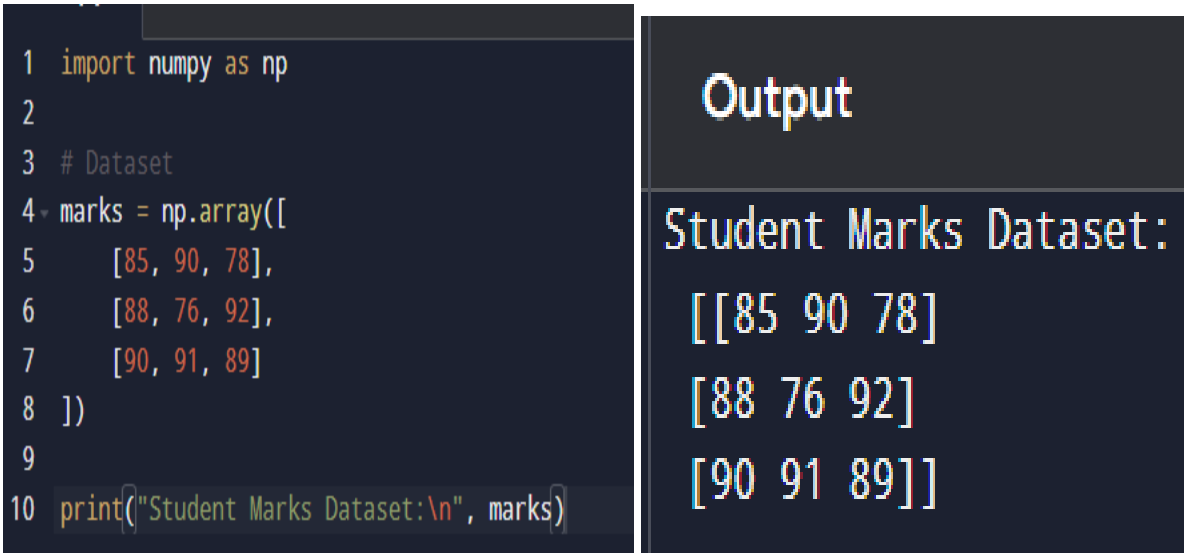
Lab Assignment: Prepare/Take dataset for any real life application. Read a dataset into an array. Perform following operations on it as:

- Perform all matrix operations
- Horizontal and vertical stacking of Numpy Arrays - Custom sequence generation - Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
- Copying and viewing arrays

```python
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

print("Array A:", a)
print("Array B:", b)

print("Addition:", a + b)
print("Subtraction:", b - a)
print("Multiplication:", a * b)
print("Division:", b / a)

print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))

print("Square Root:", np.sqrt(a))
print("Power:", np.power(a, 2))

print("Index of 3:", np.where(a == 3))
print("Sorted Array:", np.sort(b))

c = np.array([1,2,3,4,5,6])
print("Reshaped Array:\n", c.reshape(2,3))
```

```
Array A: [1 2 3 4 5]
Array B: [10 20 30 40 50]
Addition: [11 22 33 44 55]
Subtraction: [ 9 18 27 36 45]
Multiplication: [ 10  40  90 160 250]
Division: [10. 10. 10. 10. 10.]
Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Square Root: [1.         1.41421356 1.73205081
Power: [ 1  4  9 16 25]
Index of 3: (array([2]),)
Sorted Array: [10 20 30 40 50]
Reshaped Array:
 [[1 2 3]
 [4 5 6]]
```

Self Study Assignment: For any real life application, perform advanced data operations such as image as array and image manipulations.



```
1  import numpy as np
2
3  # Dataset
4  marks = np.array([
5      [85, 90, 78],
6      [88, 76, 92],
7      [90, 91, 89]
8  ])
9
10 print("Student Marks Dataset:\n", marks)
```

```
Output

Student Marks Dataset:
 [[85 90 78]
 [88 76 92]
 [90 91 89]]
```

PRACTICAL NO.04 Practice Lab Assignment: 1. Perform all the pandas operations

main.py

```python
import pandas as pd

data = {
    "Name": ["Amit", "Rahul", "Sneha"],
    "Age": [21, 22, 20],
    "Marks": [85, 90, 88]
}

df = pd.DataFrame(data)

print("DataFrame:\n", df)

print("\nHead:\n", df.head())
print("\nTail:\n", df.tail())

print("\nInfo:")
print(df.info())

print("\nDescribe:\n", df.describe())

print("\nNames:\n", df["Name"])
print("\nMarks > 85:\n", df[df["Marks"] > 85])

print("\nSorted by Marks:\n", df.sort_values("Marks"))

df["Result"] = ["Pass", "Pass", "Pass"]
print("\nAfter Adding Column:\n", df)

df.drop("Result", axis=1, inplace=True)
print("\nAfter Deleting Column:\n", df)
```

```
DataFrame:
     Name  Age  Marks
0   Amit   21     85
1  Rahul   22     90
2  Sneha   20     88

Head:
     Name  Age  Marks
0   Amit   21     85
1  Rahul   22     90
2  Sneha   20     88

Tail:
     Name  Age  Marks
0   Amit   21     85
1  Rahul   22     90
2  Sneha   20     88

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   Marks   3 non-null      int64
dtypes: int64(2), object(1)
memory usage: 204.0+ bytes
None

Describe:
         Age      Marks
count   3.0   3.000000
mean   21.0  87.666667
std     1.0   2.516611
min    20.0  85.000000
25%    20.5  86.500000
```

Lab Assignment:

Read any real life dataset. Store the data into Data Frames. Identify 10 grains for the given dataset. Implement all 20 grains using Pandas methods. The Sample Grains for Sales Dataset as:

- Which was the best month for sales? How much was earned that month?
- Which product sold the most? Why do you think it did?
- Which city sold the most products?
- What Products are most often sold together?

```python
import pandas as pd

# Sample Sales Dataset
data = {
    "Month": ["Jan", "Feb", "Mar", "Jan", "Feb"],
    "Sales": [1000, 2000, 3000, 1500, 2500]
}

df = pd.DataFrame(data)

monthly_sales = df.groupby("Month")["Sales"].sum()

best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print("Best Month:", best_month)
print("Sales Earned:", highest_sales)
```

PRACTICAL NO.05

Practice Lab Assignment:

1. Install MatPlotLib library. Draw basic graphs for sales dataset using MatPlotLib

```python
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [1000, 1500, 2000, 1800, 2500]

plt.plot(months, sales, marker='o')

plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.grid(True)
plt.show()
```

Lab Assignment:

For Titanic dataset, Perform data analysis. Identify 5 grains for a given dataset. Perform a data visualization for those grains using the MatPlotLib library. (Use any 5 different graphs with proper title, legends, axis names, etc.to map identified grains)

```python
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Survived": [1, 0, 1, 1, 0, 0, 1]
}

df = pd.DataFrame(data)

survival = df["Survived"].value_counts()

plt.bar(["Not Survived", "Survived"], survival)

plt.title("Titanic Survival Count")
plt.xlabel("Status")
plt.ylabel("Passengers")

plt.show()
```

Self Study Assignment:

1. For any real life application, perform advanced graphs for data visualization such as Span Selector, Broken Barh-Broken Horizontal Bar plot, Watermarking Images with Matplotlib.

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import SpanSelector

x = np.arange(0, 10, 0.01)
y = np.sin(x)

fig, ax = plt.subplots()
ax.plot(x, y)

plt.title("Span Selector Example")
plt.show()
```